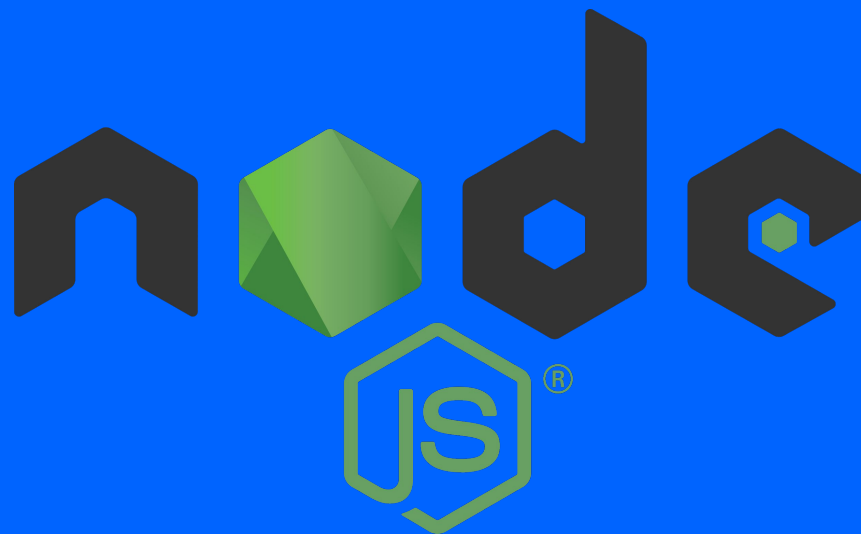
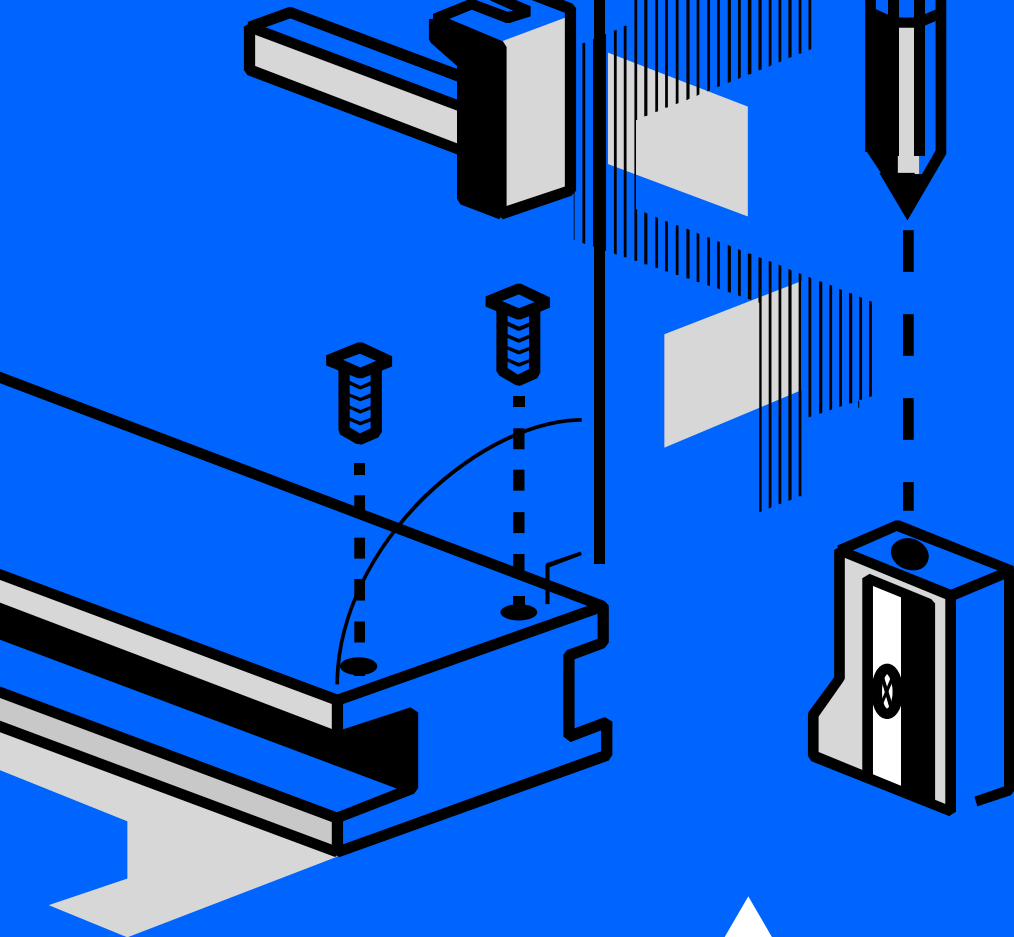




מבנה פרויקט מודולרי יקיר אלביליה

המכללה
הטכנולוגית
באר שבע

בשיתוף WORLD ORT
קדימה מדע



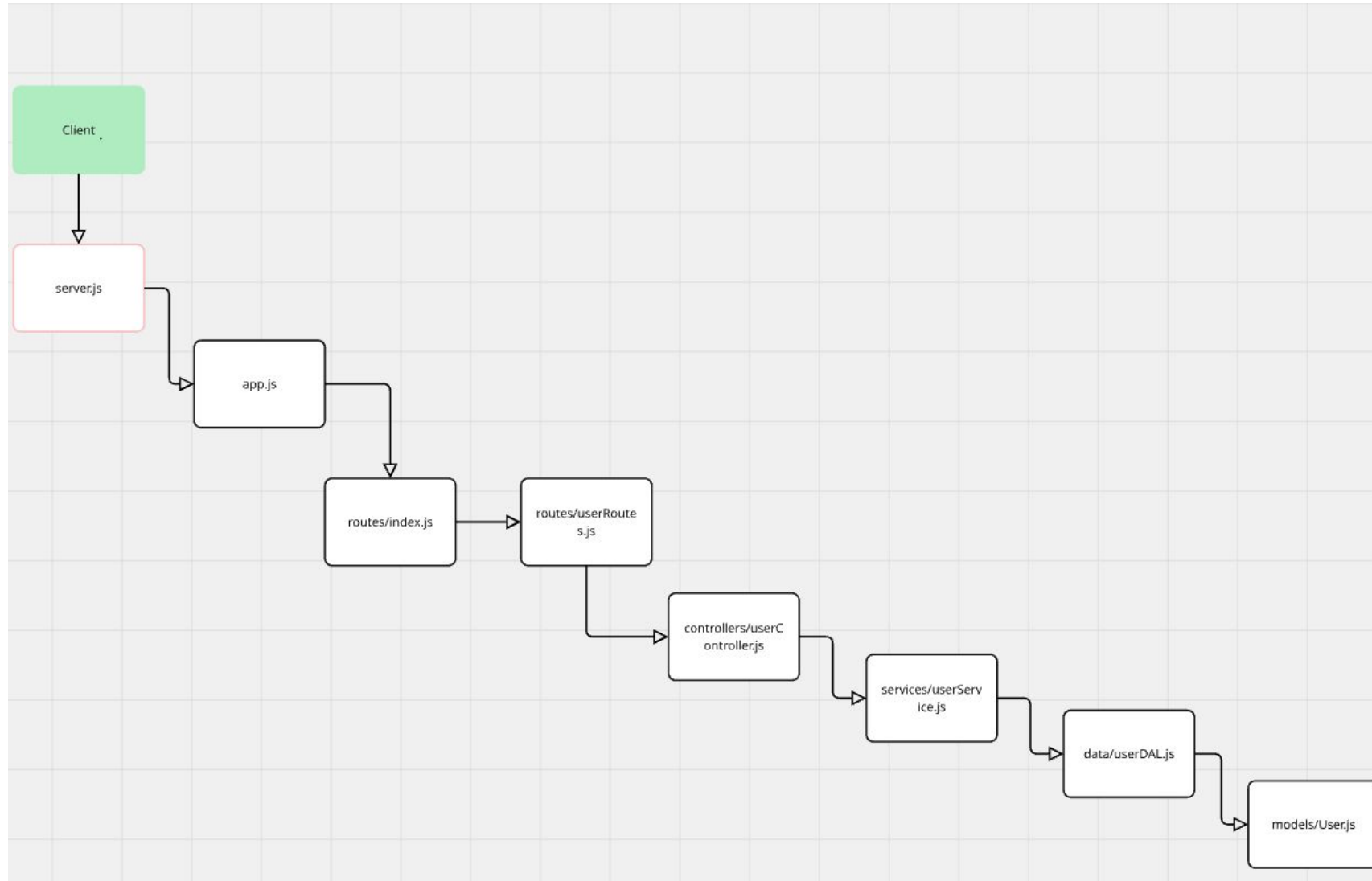
מה נלמד היום?

- מהי ארכיטקטורת שרת מודולרית
- מהן שכבות השרת ואיך זורם המידע
- דוגמת פרויקט Express מלא
- תרשימי זרימה להבנה ויזואלית

סקירה כללית על שכבות השרת

שכבה (Layer)	תפקיד במערכת	
Entry Point	קובץ ההתנעה הראשי (server.js / app.js)	1
Routes	הגדרת נתיבי ה-API של השרת	2
Middleware	פונקציות עזר הפועלות לפני או אחרי הטיפול בבקשה	3
Controllers	ניתוב והגדרת הפעולה הנדרשת עבור כל בקשה	4
Services	הלוגיקה העסקית והחשובים של האפליקציה	5
Models	הגדרת מבנה הנתונים מול מסד הנתונים (DB)	6
Infrastructure	חיבור לתשתיות ושירותים חיצוניים (DB, מיילים)	7
Utils / Helpers	ארגז כלים: פונקציות עזר קטנות וגנריות	8
Configuration	ניהול משתני סביבה והגדרות כלליות	9

תרשים זרימה: זרימת הבקשה



שכבת הכניסה – Entry Point



app.js / server.js

מטרה:

נקודת ההתחלה של האפליקציה – יצירת מופע של Express, חיבור למסד הנתונים, והאזנה לבקשות.

תפקידים:

- אתחול האפליקציה
- שימוש ב-middlewares כלליים
- טעינת הנתיבים (routes)
- חיבור למסד נתונים

server.js

```
// server.js

import app from './app.js';

import config from 'config/index.js';

const PORT = config.PORT || 3000;

app.listen(PORT, () => {

  console.log(`Server running on port ${PORT}`);

});
```

app.js

```
// app.js

import express from 'express';
import routes from './routes/index.js';

const app = express();
app.use(express.json());

app.use('/', routes);

export default app;
```

שכבת הראוטינג – Routes



מטרה:

הגדרת מסלולי ה-API (endpoint) לכל משאב במערכת, וחיבורם לבקרים Controllers

תפקידים:

- הגדרת endpoints שונים
- חלוקת נתיבים לפי תחומים (users, products וכו')
- הפניה לבקר הרלוונטי

routes/index.js

```
import {Router} from 'express';  
  
import userRoutes from './userRoutes.js';  
  
const router = Router();  
router.use('/users', userRoutes);  
  
export default router;
```

routes/ userRoutes.js

```
import {Router} from 'express';  
  
import {registerUser,loginUser,} from '../controllers/userController.js'  
  
const router = Router();  
  
router.post('/register', registerUser);  
router.post('/login', loginUser);  
  
export default router;
```

שכבת המידלור – Middleware



middleware/

מטרה:

טיפול בבקשות/תגובות בין קבלתן לבין ההגעה לבקרים.

תפקידים:

- אימות משתמשים auth
- טיפול בשגיאות
- לוגים
- עיבוד נתונים לפני השלב הלוגי

authMiddleware.js

```
export const authenticate = (req, res, next) => {  
  
  const token = req.headers.authorization;  
  
  if (!token) return res.status(401).json({ message: 'Unauthorized' });  
  
  // JWT תהליך בדיקת ...  
  
  next();  
}
```

שכבת הבקרים – Controllers



controllers/

מטרה:

לוגיקת התגובה לבקשות, HTTP – אחראית על התקשורת עם שכבות נמוכות יותר.

תפקידים:

- ניתוח הבקשה
- הפעלת שירותים services
- החזרת תגובה ללקוח

controllers/userController.js

```
import userService from '../services/userService.js';

export const registerUser = async (req, res) => {

  try {

    const result = await userService.register(req.body);

    res.status(201).json(result);

  } catch (error) {

    console.error('Error registering user:', error);

  }
};
```

שכבת השירותים – Services



מטרה:

אחראית על הלוגיקה העסקית – פעולות מורכבות או קריאות מול הנתונים.

תפקידים:

- ניהול תהליכים עסקיים
- קריאה למודלים או ל-DAL
- אבסטרקציה לבקרים

services/userService.js

```
import userDal from '../dal/userDal.js';

import hashPassword from '../utils/hashPassword.js';

const register = async (userData) => {

  try {

    const hashed = await hashPassword(userData.password);

    return await userDal.create({ ...userData, password: hashed });

  } catch (error) {

    console.error('Error in user registration:', error);

    throw error; }

};

export default { register };
```

שכבת הגישה לנתונים – DAL



מטרה:

ממשק גישה ל-DB – כל הגישה הישירה למסד מרוכזת כאן.

תפקידים:

- ביצוע שאילתות
- הפשטת המידע ממסד הנתונים

dal/userDal.js

```
import User from '../models/User.js';

const create = async (userData) => {

  try {

    return await User.create(userData);

  } catch (error) {

    console.error('Error creating user:', error);

    throw error;

  }

};

export default { create };
```

שכבת המודלים – Models



מטרה:

הגדרת מבני הנתונים מול מסד הנתונים – Schema של MongoDB / SQL

תפקידים:

- הגדרת שדות וסוגים
- הטמעת ולידציות

models/User.js

```
import mongoose from 'mongoose';

const userSchema = new mongoose.Schema({
  name: String,
  email: { type: String, unique: true },
  password: String,
});

export default mongoose.model('User', userSchema);
```

שכבת הכלים – Utilities

/utils 

מטרה:

פונקציות עזר כלליות – ללא תלות בלוגיקה עסקית.

תפקידים:

- עיבוד מחרוזות, תאריכים
- הצפנה / hashing
- שליחת מיילים

utils/hashPassword.js

```
import bcrypt from 'bcrypt';

export default async function hashPassword(password) {

  const salt = await bcrypt.genSalt(10);

  return bcrypt.hash(password, salt);

}
```

שכבת הקונפיגורציה – Configuration

 config/

מטרה:

שמירה על משתנים רגישים והגדרות סביבת העבודה.

תפקידים:

- ניהול משתני סביבה
- עבודה לפי development, production
- אבטחת נתונים רגישים

config/index.js

```
import 'dotenv/config';

export default {
  port: process.env.PORT || 3000,
  mongoURI: process.env.MONGO_URI,
  jwtSecret: process.env.JWT_SECRET
};
```

config/db.js

המכללה
הטכנולוגית
באר שבע



בשיתוף WORLD ORT
קדימה מדע

```
import mongoose from 'mongoose';

import config from './config.js';

const connectDB = async () => {

  try {

    const conn = await mongoose.connect(config.mongoURI);

    console.log(`MongoDB Connected: ${conn.connection.host}`);

    return conn;

  } catch (error) {

    console.error('Error connecting to MongoDB:', error.message);

    process.exit(1);

  }

};

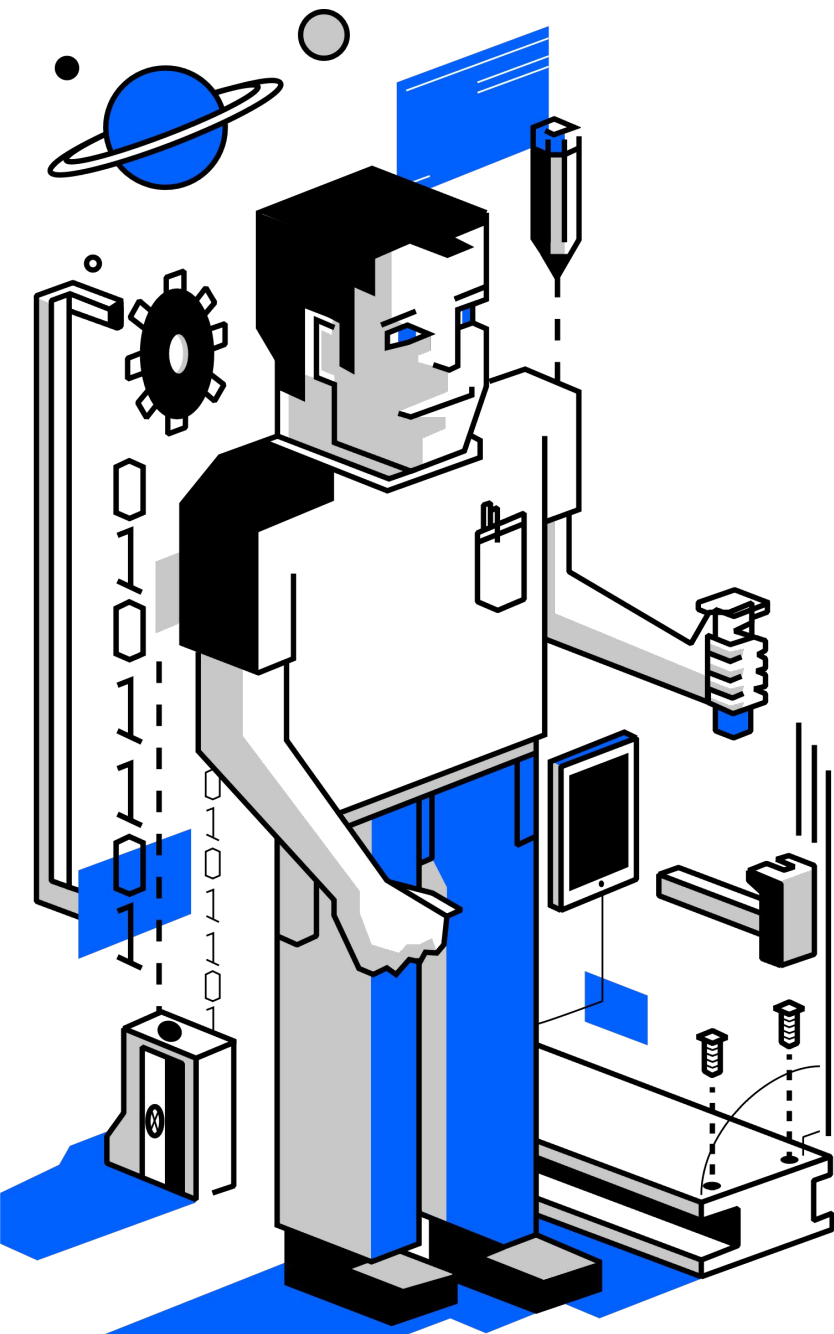
export default connectDB;
```

סיכום – מבנה שכבות בפרויקט Node.js

השכבות שראינו:

- 1. **Entry Point** – אתחול האפליקציה
- 2. **Routes** – מיפוי נתיבים
- 3. **Middleware** – טיפול בדרך
- 4. **Controllers** – תגובה לבקשות
- 5. **Services** – לוגיקה עסקית
- 6. **DAL** – גישה לנתונים
- 7. **Models** – מבנה הנתונים
- 8. **Config** – משתני סביבה
- 9. **Utils** – פונקציות עזר

- הפרדת שכבות יוצרת קוד נקי, גמיש ומאורגן
בזכות חלוקה ברורה, כל חלק במערכת יודע את מקומו
ואחריותו.
- יתרונות הגישה המודולרית:
 - תחזוקה קלה – שינוי בשכבה אחת כמעט ולא משפיע על אחרות
 - שימוש חוזר – אפשר להשתמש בפונקציות בשכבות שונות בלי כפילויות
 - בדיקות נוחות – ניתן לבדוק כל שכבה בנפרד (Unit Tests)
 - סקלאביליות – קל להוסיף פיצ'רים מבלי להרוס את הקיים



שאלות

המכללה
הטכנולוגית
באר שבע

בשיתוף WORLD ORT
קדימה מדע

